SOP: B1 Version-Aware Retrieval, Track B (Kafka runbooks)

Standard operating procedure and runbook: build, test and recreate the B1 experiment
Modern AI Pro · Level 2 · Track B · RAG Design · Owner: Shreenath Shetty

Version 6 · 2026-10-18 · all 12 steps complete · public on GitHub (section 11) · every command in Appendix A

Owner	Shreenath Shetty (individual project)
Course	Modern AI Pro · Level 2 · Track B · RAG Design · project B1
Guide	`project_10082026/B1_Version_Aware_Retrieval_Step_by_Step_Guide.pdf`
Code	`project_10082026/track-b-kafka/` · branch `track-b`
Evidence	`project_10082026/b1-tracking/track-b/runs/`
Last updated	2026-10-18 (all 12 steps complete; published to GitHub)
GitHub	https://github.com/shreenathacc22/L2-RAG-B1-track-b-kafka-version-aware (public) · branches `main`, `track-b-kafka-version-aware`, `rag-deployment`

0. Status at a glance

Step	What	Status	Commit / evidence
1	Team setup	N/A (individual)	—
2	Run unchanged `simple_rag`	✅ Done	`runs/step02-baseline-unchanged/`
3	Versioned corpus (16 runbooks) and data checks	✅ Done	`636c6a7` · `runs/step03-corpus/`
4	Metadata manifest	✅ Done	`e480d14` · `runs/step04-manifest/`
5	Golden set (21 cases)	✅ Done	checks `24f4456` · `runs/step05-golden/`
6	Freeze golden set in Git	✅ Done	`2cff45a` · `runs/step06-freeze/`
7	Evaluator and BEFORE run (filter OFF)	✅ Done: 48.8% superseded	`8c97952` · `runs/step07-before/`
8	Version metadata into Chroma	✅ Done: 17/17 chunks tagged; OFF unchanged	`65ee770` · `runs/step08-chroma-metadata/`
9	Query-time filter	✅ Done	`3e269fd` · `runs/step09-10-filter/`
10	OFF/ON switch	✅ Done: OFF = baseline, ON preview 0%	`3e269fd` · `runs/step09-10-filter/`
11	AFTER run (filter ON)	✅ Done: 0.0% superseded	`fbc4a92` · `runs/step11-after/`

Step	What	Status	Commit / evidence
12	Ablation table and conclusion	✅ Done	`896d73b` · `runs/step12-report/`

0.1 Recreate without Claude: start here

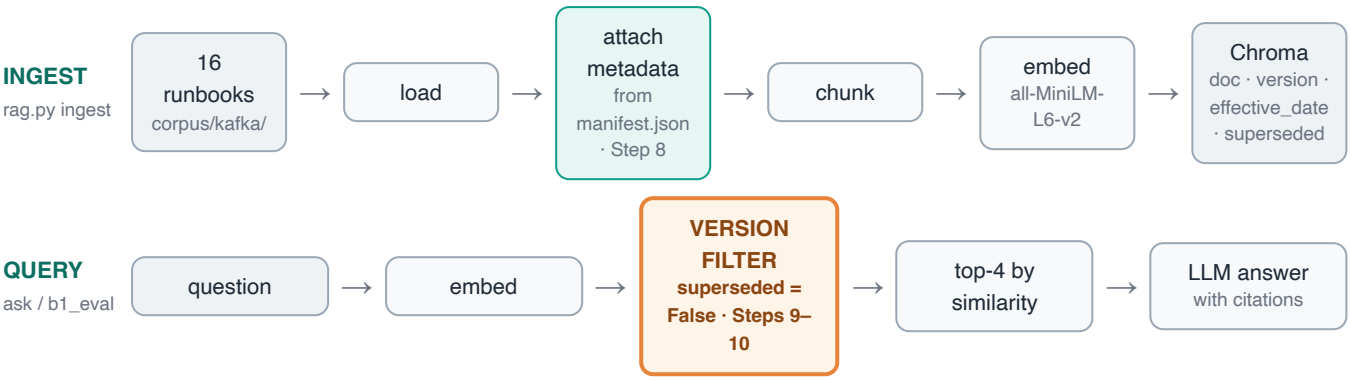
Everything below can be rebuilt and tested from a terminal, with no AI assistant.

- **From GitHub:** section 11 has the clone commands for every branch.
- **Fastest:** one command, Appendix A.1. It rebuilds Steps 0–12 in a new folder (about 2 minutes) and proves the result is identical.
- **Step by step:** Appendix A.2 lists every command in order, with what you should see.
- **Proof:** Appendix A.3 lists the fingerprints a correct rebuild must match.

1. The idea in five lines

1. Companies keep old and new versions of the same runbook. They read almost identically.
2. RAG retrieves by meaning (similarity), so it can't tell which version is current. It often returns the old one, or both.
3. The LLM then answers with an outdated threshold, port or team. That's a real incident risk.
4. **Fix:** tag every chunk with `version`, `effective_date` and `superseded`, and have the retriever exclude `superseded=true` *before* ranking.
5. **Proof:** run the same frozen test cases with the filter OFF and then ON. Superseded chunks should drop to 0%, and exact-value lookups should stay flat.

2. The flow



Orange = the one new stage B1 adds. Green = the metadata it needs. Grey = unchanged lab code. The filter switches OFF (baseline) or ON (fixed) for the ablation.

Every box already exists in `simple_rag/rag.py` except **VERSION FILTER**. B1 adds that one stage.

3. Where everything is

```
project_10082026/
├── .env                          class proxy key (never commit, never print)
├── B1_..._Guide.pdf              the assignment
├── mai-practitioner-labs/.venv/  shared Python 3.12 environment
├── track-b-kafka/                ◄── THE CODE (git, branch track-b)
│   └── simple_rag/
│       ├── rag.py                pipeline; Step 7 settings, Step 8 metadata, Steps 9-10 filter +
switch
│       ├── b1_check.py           data tests (Steps 3-5) + --chroma (Step 8) ✅
│       ├── b1_inspect_chroma.py  look inside Chroma + filter demo (Step 8) ✅
│       ├── b1_eval.py            BEFORE/AFTER evaluator (Steps 7, 11)  ✅
│       ├── golden_kafka.jsonl    21 test cases (Step 5; frozen in Step 6) ✅
│       ├── corpus/kafka/
│       │   ├── <subject>.md          8 current runbooks                ✅
│       │   ├── <subject>-<year>.md   8 old runbooks                    ✅
│       │   └── manifest.json     version metadata (Step 4)             ✅
│       └── results/              baseline_filter_off.json, version_filter_on.json,
│                                 ablation_summary.md (Step 7 ✅; 11, 12 □)
└── b1-tracking/track-b/          ◄── THIS SOP AND THE EVIDENCE
    ├── SOP.md                    this document (source; edit this one)
    ├── docs/                     PDF of this SOP + the flow deck (.pptx)
    ├── tools/recreate_track_b.sh ONE COMMAND: rebuilds Steps 0-6 and verifies them
    ├── tools/make_corpus.py      regenerates the 16 runbooks + manifest.json
    ├── tools/make_golden.py      regenerates the 21 golden cases
    ├── tools/source/             committed sources: Steps 3-7 top level, step08/, step10/
    ├── tools/docs/               scripts that rebuild the PDF and the deck
    └── runs/<step>/              logs from every run
```

4. One-time setup

Skip this if `mai-practitioner-labs/.venv` already exists. (Check with `ls ~/Desktop/AI-Architect_L2_Aug_2026/project_10082026/mai-practitioner-labs/.venv/bin/python`.)

```
B=~/Desktop/AI-Architect_L2_Aug_2026/project_10082026
cd $B/mai-practitioner-labs
uv venv --python 3.12 .venv     # 3.12: newer Pythons lack wheels on Intel macOS
source .venv/bin/activate
uv pip install -r requirements.txt "torch==2.2.2" "numpy<2" "transformers<5" "sentence-transformers<6"
```

Get the code (only if `track-b-kafka/` is missing):

```
cd $B
git clone https://github.com/balajivis/mai-practitioner-labs.git track-b-kafka
cd track-b-kafka && git switch -c track-b
cp ../.env .env && chmod 600 .env
```

Start every working session with:

```
B=~/Desktop/AI-Architect_L2_Aug_2026/project_10082026
cd $B/track-b-kafka
source $B/mai-practitioner-labs/.venv/bin/activate
which python    # must end in project_10082026/mai-practitioner-labs/.venv/bin/python
```

---

5. Procedure

Each step has the same parts: **Why**, **Do**, **Expect**, **Verify**, **Evidence**.

Step 2: Prove the unchanged lab works ✅

Why: If something breaks later, you need to know it worked before you touched it.

Do:

```
python simple_rag/rag.py ingest                          # Aventro corpus (the lab default)
python simple_rag/rag.py ask "how do I tow my car?"      # needs the class proxy (.env)
python simple_rag/rag.py eval                            # the lab's own 6 cases
```

Expect: All three exit without error. `ingest` stores ~196 vectors. `ask` prints a cited answer. `eval` prints a scorecard.

Verify: The scorecard is **retrieval 2/6 · answer 2/6**. That's the lab's designed starting point, not a fault.

Evidence (2026-10-17): `runs/step02-baseline-unchanged/` contains `ingest.log`, `ask.log` (21 s) and `eval.log` (60 s).

Step 3: Build the versioned corpus ✅

Why: You need old/current pairs that are near-identical in wording but conflict in facts. Only then does similarity search get fooled, and only then can the fix be measured.

What was built: 8 subjects × 2 versions = 16 runbooks in `simple_rag/corpus/kafka/`, for a fictional company, *Halcyon*. It's not Meridian or Aventro, which the guide forbids.

Subject	Current (file · version · date)	Old (file · version · date)	Key conflict: old → current
Consumer lag	`consumer-lag-alerting.md` v2 2026-03-01	`-2024.md` v1 2024-06-01	page at 100,000 msgs → 10 min time lag
Retention	`topic-retention.md` v3 2026-01-15	`-2024.md` v2 2024-09-01	7 days (604800000) → 3 days (259200000)
Rolling restart	`broker-rolling-restart.md` v4 2026-04-01	`-2025.md` v3 2025-03-10	wait 5 → 10 min; controller last
Listeners/auth	`listeners-and-auth.md` v3 2026-02-01	`-2024.md` v1 2024-03-01	9092 PLAINTEXT → 9094 SASL_SSL
Topic creation	`topic-creation-standards.md` v3 2026-02-15	`-2024.md` v2 2024-05-20	6 → 12 partitions; min ISR 1 → 2
Disk capacity	`disk-capacity.md` v3 2026-05-01	`-2025.md` v2 2025-02-01	page at 90% → 85%
Schema registry	`schema-registry.md` v2 2026-03-20	`-2024.md` v1 2024-07-15	port 8081 → 8085; BACKWARD → FULL_TRANSITIVE
On-call escalation	`oncall-escalation.md` v3 2026-06-01	`-2025.md` v2 2025-01-10	ack 15 → 5 min; `kafka-ops` → `streaming-platform-prod`

Each pair has 5–9 conflicting values. Some values are deliberately the **same** in both versions (replication factor 3, `/var/lib/kafka`, `auto.create.topics.enable=false`). Those are used later for exact-string tests that the filter must not break.

Design rules (enforced by `b1_check.py`):

- Naming: `<subject>.md` is current, `<subject>-<year>.md` is old.
- Old runbooks have **no "superseded" banner**. They read as live documents. Status lives only in metadata (Step 4).
- No front-matter. Metadata goes in `manifest.json` so it isn't embedded with the text.
- The header line is `Version N · Effective YYYY-MM-DD · Owner: ...`.

Do (only to recreate from scratch):

```
python $B/b1-tracking/track-b/tools/make_corpus.py simple_rag/corpus/kafka   # also writes
manifest.json (Step 4)
git status --short simple_rag/corpus/kafka    # empty = regenerated byte for byte
```

Verify:

```
python simple_rag/b1_check.py
diff simple_rag/corpus/kafka/disk-capacity-2025.md simple_rag/corpus/kafka/disk-capacity.md   # only
facts differ
```

Expect:

```
OK  16 runbooks
OK  8 subjects, each with a current <subject>.md and an old <subject>-<year>.md
OK  every file has a Version/Effective header, no status banner, no front-matter
OK  current version number > old version number in every pair
    pair similarity (current vs old): min 0.867  max 0.969
OK  every pair is near-duplicate (>= 0.85)
    old version outranks current for 4/8 plain questions
OK  the trap is real
PASS
```

How to read the trap result: For every plain question, the old and current versions rank #1 and #2, in either order. The retriever returns the top 4, so **both** get retrieved and about half the context is stale. That's the problem the filter must remove.

Evidence: commit `636c6a7`, `runs/step03-corpus/b1_check.log`.

Step 4: Metadata manifest ✅

Why: The retriever needs a machine-readable answer to "is this file current?". Similarity can't provide it; metadata can.

What was built: `simple_rag/corpus/kafka/manifest.json`, written by the same generator as the runbooks, so text and metadata can't drift apart.

```
"files": {
  "disk-capacity.md":      {"doc": "Kafka Broker Disk Capacity Runbook", "version": 3,
"effective_date": "2026-05-01", "superseded": false},
  "disk-capacity-2025.md": {"doc": "Kafka Broker Disk Capacity Runbook", "version": 2,
"effective_date": "2025-02-01", "superseded": true},
  ...
},
"conflicts": {
  "Kafka Broker Disk Capacity Runbook": [
    {"field": "page", "old": "90%", "current": "85%"}, ...
  ]
}
```

Field	Type	Meaning
`doc`	string	Title shared by every version. This is what links a pair
`version`	integer	Higher is newer
`effective_date`	`YYYY-MM-DD`	Date this version took effect; matches the file's header
`superseded`	boolean	`true` once a newer version exists. **The filter in Step 9 uses only this field**

`conflicts` lists all 64 values that differ between versions. It's a reference for writing golden cases and is **never** given to the retriever.

Why not front-matter? Track A put metadata inside each `.md`. Track B keeps documents clean, so the text that gets embedded is exactly what a reader sees. The metadata travels separately and is attached to each chunk at ingest (Step 8).

Verify:

```
python simple_rag/b1_check.py --no-trap
```

Expect:

```
[Step 4] manifest
OK  manifest lists exactly the 16 runbooks on disk
OK  every entry has exactly 4 fields with the right types; versions and dates match the files
OK  8 docs, each with exactly 1 current file that has the highest version and latest date (8 current,
8 superseded)
OK  all 64 conflict values found in their files
    55/64 old values never appear in the current file — usable as must_not_contain in Step 5
PASS
```

Negative test (proves the checks work): On purpose, mark an old file current, make a version a string and break a date. Run the checks, then restore:

```
M=simple_rag/corpus/kafka/manifest.json; cp $M /tmp/manifest.bak
python - <<'PY'
import json; p="simple_rag/corpus/kafka/manifest.json"; m=json.load(open(p))
m["files"]["disk-capacity-2025.md"]["superseded"]=False
m["files"]["topic-retention.md"]["version"]="3"
m["files"]["schema-registry.md"]["effective_date"]="2026-3-20"
json.dump(m,open(p,"w"),indent=2)
PY
python simple_rag/b1_check.py --no-trap     # expect FAIL (5), exit code 1
cp /tmp/manifest.bak $M                     # restore
python simple_rag/b1_check.py --no-trap     # back to PASS
```

Evidence: commit `e480d14`; `runs/step04-manifest/b1_check.log` (PASS) and `negative_test.log` (FAIL 5, as intended).

Step 5: Golden set ✅

Why: These are the exam questions. They're written *before* the fix, so they test what matters and not what the system already happens to do.

What was built: `simple_rag/golden_kafka.jsonl`, with 21 cases, one JSON object per line. It follows the class schema in `golden/SCHEMA.md`. Ids are `kafka-NNN`, which keeps them apart from the lab's `av-NNN` cases.

Anatomy of one case:

```
{"id": "kafka-001", "type": "superseded", "class": "accuracy", "bucket": "version_conflict",
 "question": "Broker 4's data volume just hit 87%. Should that have paged someone?",
 "expect": {
   "behavior": "answer",                                                    // answer | refuse
   "facts": ["Yes. The page threshold is 85% used, so 87% pages the on-call"],
   "must_cite":     [{"doc": "Kafka Broker Disk Capacity Runbook", "version": 3}],  // must be
retrieved
   "must_not_cite": [{"doc": "Kafka Broker Disk Capacity Runbook", "version": 2}],  // must NOT be
retrieved
   "must_not_contain": ["90%"],                                             // stale literal; answer
must not say it
   "forbidden": ["Saying 87% is below the page threshold"]},                // for a human or LLM
judge
 "rubric": "...", "why": "..."}
```

How each type is scored (by `b1_eval.py` in Step 7; retrieval only, no LLM):

Type	Count	Passes when
`superseded`	9	Every `must_cite` doc retrieved **and** no `must_not_cite` doc retrieved
`exact_string`	7	The exact value in `facts` appears in the retrieved text
`multi_hop`	3	Both `must_cite` docs retrieved, no `must_not_cite` doc retrieved
`unanswerable`	2	Answer level only (LLM): says "not documented", invents nothing

Design techniques used:

- **Between-threshold questions.** 87% disk, or 8 minutes of lag, falls between the old and new rule, so the two versions give *opposite* answers. A stale answer can't pass by luck.
- **Stale literals.** `must_not_contain` holds the old value (`90%` , `604800000` , `9092` , `8081` , `kafka-ops` ...). A plain string match catches a stale answer, with no judge needed.
- **Traps in the answer.** In `kafka-003` and `kafka-008` , the stale number also appears in the *current* doc with a different meaning (7 days = self-service max; 15 min = secondary escalation). Those cases use `forbidden` and not `must_not_contain` , so a correct answer isn't failed.
- **Version-neutral exact values.** 5 of the 7 `exact_string` values (`/var/lib/kafka` , `orders.v1-value` ...) are identical in both versions. The filter shouldn't change their score, which is the "stay flat" proof.
- **Engineer wording.** Questions are phrased the way someone on call would ask, not copied from the runbook.

All 21 cases:

id	type	question	expected	must_cite	must_not_contain
kafka-001	superseded	Broker 4's data volume just hit 87%. Should that have paged someone?	Yes. The page threshold is 85% used, so 87% pages the on-call	Broker Disk Capacity v3	`90%`
kafka-002	superseded	Our payments consumers are about 8 minutes behind, roughly 30,000 messages. Does that page anyone?	No page; it is a warning. Warning is above 5 minutes of time lag, the page is above 10 minutes	Consumer Lag Alerting v2	`100,000 messages` , `50,000 messages`
kafka-003	superseded	I created a topic and never set retention. How long will messages stay around?	3 days (retention.ms 259200000)	Topic Retention Policy v3	`604800000`
kafka-004	superseded	Broker 2 just rejoined after its restart. How long until I can bounce broker 3?	10 minutes after the broker rejoins, and only once the pre-check passes again	Broker Rolling Restart Procedure v4	`5 minutes`
kafka-005	superseded	What host and port do I put in my new producer's bootstrap config?	kafka.prod.halcyon.internal:9094, SASL_SSL	Listeners and Client Authentication v3	`9092` , `kafka-prod.halcyon.internal`
kafka-006	superseded	I'm spinning up a topic with the standard settings. How many partitions and what min ISR?	12 partitions; min.insync.replicas 2	Topic Creation Standards v3	—
kafka-007	superseded	What URL should my serializer use to reach the schema registry?	https://schemas.prod.halcyon.internal:8085	Schema Registry Operations v2	`8081`

id	type	question	expected	must_cite	must_not_contain
kafka-008	superseded	I'm primary on-call and Kafka just paged me. How long do I have to acknowledge?	5 minutes	On-Call Escalation Policy v3	—
kafka-009	superseded	Which PagerDuty service do I trigger for a production Kafka problem?	streaming-platform-prod	On-Call Escalation Policy v3	`kafka-ops`
kafka-010	exact_string	Can producers auto-create topics on prod? What's the broker setting?	auto.create.topics.enable=false	Topic Creation Standards v3	—
kafka-011	exact_string	Which mount should I watch for Kafka broker disk alerts?	/var/lib/kafka	Broker Disk Capacity v3	—
kafka-012	exact_string	What command tells me whether a lagging group's consumer pods are actually running?	kubectl get pods -l app=	Consumer Lag Alerting v2	—
kafka-013	exact_string	What's the registry subject for the value schema of topic orders.v1?	orders.v1-value	Schema Registry Operations v2	—
kafka-014	exact_string	My app can't connect and logs a node -1 error. What's the exact message, and what does it usually mean?	Connection to node -1 could not be established	Listeners and Client Authentication v3	—
kafka-015	exact_string	What's the change template ID for a broker rolling restart?	STD-KAFKA-ROLL	Broker Rolling Restart Procedure v4	—
kafka-016	exact_string	Where in Vault do I find my app's Kafka client credentials?	secret/kafka/clients/	Listeners and Client Authentication v3	`svc-kafka`
kafka-017	multi_hop	Disk on broker 3 just crossed the page line. What is that line, and which PagerDuty service gets the page?	85%; streaming-platform-prod	Broker Disk Capacity v3, On-Call Escalation Policy v3	`90%`, `kafka-ops`

id	type	question	expected	must_cite	must_not_contain
kafka-018	multi_hop	New topic, all defaults: how many partitions does it get and how long is data kept?	12 partitions; 3 days	Topic Creation Standards v3, Topic Retention Policy v3	`604800000`
kafka-019	multi_hop	I'm rolling the brokers for a TLS change. How long do I wait between brokers, and which client port should be healthy afterwards?	10 minutes; 9094	Broker Rolling Restart Procedure v4, Listeners and Client Authentication v3	`9092`
kafka-020	unanswerable	What's the largest message size our brokers will accept?	The runbooks do not state a maximum message size	—	—
kafka-021	unanswerable	How many brokers are in the production cluster?	The runbooks do not state the broker count	—	—

Verify:

```
python simple_rag/b1_check.py --no-trap
```

Expect:

```
[Step 5] golden set
OK  21 cases, unique ids, exact_string 7, multi_hop 3, superseded 9, unanswerable 2
OK  every must_cite is current, every must_not_cite is superseded
OK  every must_not_contain is in the old file only; every exact value is verbatim in its file
OK  unanswerable cases cite nothing and expect a refusal; multi_hop cases cite 2 docs
OK  no question copies 7+ consecutive words from the runbook it targets
    current runbooks covered by must_cite: 8/8
PASS
```

What the checks protect against:

Check	Mistake it catches
`must_cite` is a current file	A case that rewards the stale version
`must_not_contain` is in the old file only	A banned string that a *correct* answer would contain, which fails good answers
Exact value verbatim in the file	A typo in the expected value; the case could never pass
Paraphrase (fewer than 7 shared words in a row)	Questions copied from the runbook, which inflate retrieval scores

The paraphrase check found a real defect in the first draft. `kafka-014` reused 8 words from the listeners runbook, so it was reworded before the freeze.

Negative test:

```
G=simple_rag/golden_kafka.jsonl; cp $G /tmp/golden.bak
python - <<'PY'
import json; p="simple_rag/golden_kafka.jsonl"; rows=[json.loads(l) for l in open(p)]
rows[0]["expect"]["must_cite"]=[{"doc":"Kafka Broker Disk Capacity Runbook","version":2}]  # cite the
OLD file
rows[2]["expect"]["must_not_contain"].append("3 days")                                     # ban the
right answer
open(p,"w").write("".join(json.dumps(r)+"\n" for r in rows))
PY
python simple_rag/b1_check.py --no-trap     # expect FAIL (4)
cp /tmp/golden.bak $G                       # restore
```

Evidence: `runs/step05-golden/b1_check.log` (PASS) and `negative_test.log` (FAIL 4, as intended).

Step 6: Freeze ✅

Why: A Git commit with a timestamp proves the questions existed before the fix. That's what makes the ablation honest.

Done (2026-10-18): The 21 cases were reviewed and committed **on their own**, before any change to `rag.py`.

Commit	`2cff45a` "Freeze B1 golden set before version-aware retrieval"
Time	2026-10-18 11:43 EDT
File	`simple_rag/golden_kafka.jsonl`, 21 lines
Fingerprint (sha1)	`850c67f2eab21aa1a6afabd481e5c3efccb7e0ba`
Last `rag.py` commit	`00fe688` (2026-08-23, upstream). No B1 change yet ✔

Verify (anytime later):

```
git log --oneline -- simple_rag/golden_kafka.jsonl    # expect exactly: 2cff45a Freeze B1 golden set
...
shasum simple_rag/golden_kafka.jsonl                  # expect
850c67f2eab21aa1a6afabd481e5c3efccb7e0ba
git log --oneline -- simple_rag/rag.py | head -1      # the first B1 commit here must be NEWER than
2cff45a
```

If the sha1 differs, the cases were edited after the freeze. Check `git diff 2cff45a -- simple_rag/golden_kafka.jsonl`.

Rule from now on: Don't edit a case between the OFF and ON runs. If a case is genuinely wrong, fix it in its own commit with the reason in the message, and re-run both OFF and ON.

Evidence: `runs/step06-freeze/freeze.log`.

Step 7: Evaluator and BEFORE run ✅

Why: You need proof the problem exists, measured on the unchanged retriever, before you fix it. An AFTER-only demo is not an ablation.

What changed in code (commit `8c97952`):

- `rag.py`: two settings, `RAG_CORPUS` and `RAG_COLLECTION`, choose the corpus and the Chroma collection, and the loader accepts a flat folder of `.md` files. **Retrieval is unchanged.** With neither setting, the lab behaves exactly as before.
- `b1_eval.py` (new): runs the 21 frozen cases through `rag.retrieve()` with k = 4. It looks up every retrieved file in `manifest.json` to see whether it's superseded, scores each case, and writes `results/baseline_filter_off.json`. No LLM, so it's free and deterministic. `--on` refuses to run until the filter exists (Steps 9–10).

Do:

```
export RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka
python simple_rag/rag.py ingest --formats md     # expect: md · 16 files ... vectors stored 17
python simple_rag/b1_eval.py --off               # expect the table below
```

Result (BEFORE, filter OFF):

Metric	Filter OFF	What it means
Superseded chunks retrieved	**41 / 84 = 48.8%**	Half of what the LLM would read is outdated
Superseded cases passed	0 / 9	The old twin was retrieved every time
Exact-string cases passed	7 / 7	Exact values are found; this must stay 7/7 after the fix
Multi-hop cases passed	0 / 3	Old twins crowd out the second document
Median retrieval latency	143.8 ms	Query embedding + Chroma search, after a warm-up

How to read the per-case lines: `FAIL kafka-001 superseded disk-capacity.md, disk-capacity-2025.md*, ...` lists the 4 retrieved files in rank order. `*` marks a superseded file.

Verify:

- Rerun `b1_eval.py --off`. Every case and every number except latency must be identical (checked 2026-10-18).
- `python simple_rag/b1_eval.py --on` should print "filter ON is not built yet".
- 17 vectors, not 16: one runbook (`consumer-lag-alerting.md`) is long enough to split into 2 chunks.

Evidence: commit `8c97952` (contains `results/baseline_filter_off.json`); `runs/step07-before/` contains `ingest.log`, `b1_eval_off.log` and the JSON.

Step 8: Version metadata into Chroma ✅

Why: The filter in Step 9 asks Chroma for `superseded = False`. That only works if every chunk stored in Chroma carries the field. This step puts it there and changes nothing else.

What changed in code (commit `65ee770`):

- `rag.py`: at ingest, `b1_metadata()` reads `manifest.json` from the corpus folder and copies `doc` (stored as `doc_title`), `version`, `effective_date` and `superseded` onto every chunk of that file, then into Chroma's metadata. **Retrieval is unchanged.** A corpus without a manifest (Aventro) gets no extra fields.
- `b1_check.py --chroma` (new check): reads the stored chunks straight from Chroma.
- `b1_inspect_chroma.py` (new, read-only): prints the store and runs a filter demo.

Do:

```
export RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka
python simple_rag/rag.py ingest --formats md          # re-ingest: metadata goes in now
python simple_rag/b1_check.py --no-trap --chroma      # expect: [Step 8] ... PASS
python simple_rag/b1_eval.py --off                    # expect: still 41/84 = 48.8%
```

Result:

Check	Result
Chunks in Chroma with all 4 fields, right types	17 / 17
Chunk values equal to `manifest.json`	17 / 17
`where={"superseded": False}`	9 chunks from 8 current files
`where={"superseded": True}`	8 chunks from 8 old files
`b1_eval.py --off` after Step 8	identical to Step 7: same 21 cases, same files, 48.8%
`--chroma` on the old (pre-Step-8) collection	FAIL (18), as it should

Why OFF must not move: metadata only *labels* chunks. Until the query uses the label (Step 9), ranking is the same. If OFF changed here, something other than metadata changed, and the ablation would be broken.

How to test inside Chroma

There are three levels, from quickest to most hands-on. Run all of them from `track-b-kafka/` with the venv active.

Level 1: the automated check (pass/fail)

```
python simple_rag/b1_check.py --no-trap --chroma
```

It prints one row per chunk (id, version, effective date, superseded), then:

```
OK  17 chunks, every one carries doc_title, version, effective_date, superseded with the right types
OK  every chunk's values match manifest.json for its file
OK  all 16 runbooks are in the collection
    where={"superseded": False} -> 9 chunks from 8 files; where={"superseded": True} -> 8 chunks from
8 files
OK  Chroma's where-filter splits the store cleanly: 8 current files vs 8 superseded files
PASS
```

Level 2: look inside, and see the filter's effect

```
python simple_rag/b1_inspect_chroma.py                                  # default question
python simple_rag/b1_inspect_chroma.py "What port is the schema registry on?"
```

It lists every chunk, counts them by `superseded`, and runs one question against Chroma twice. Measured 2026-10-18 with the default question ("Broker 4 ... 87% ..."):

```
plain top-4 (filter OFF)                      top-4 with where={"superseded": False} (filter ON)
#1  0.412  disk-capacity.md          current     #1  0.412  disk-capacity.md          current
#2  0.401  disk-capacity-2025.md  SUPERSEDED     #2  0.334  broker-rolling-restart.md current
#3  0.343  broker-rolling-restart-2025.md  SUP.  #3  0.299  topic-retention.md       current
#4  0.334  broker-rolling-restart.md current     #4  0.293  consumer-lag-alerting.md current
```

The 2025 disk runbook (the 90% rule) sits at #2 without the filter and is gone with it. That's the whole B1 idea, shown directly in the database.

Level 3: raw Chroma, no project scripts (paste into a terminal)

```
python - <<'PY'
import chromadb
col = chromadb.PersistentClient(path="simple_rag/.chroma").get_collection("kafka")
print("chunks:", col.count())
g = col.get(include=["metadatas"])
for i, m in sorted(zip(g["ids"], g["metadatas"])):
    print(i, m["version"], m["effective_date"], m["superseded"])
print("current :", sorted(col.get(where={"superseded": False})["ids"]))
print("old     :", sorted(col.get(where={"superseded": True})["ids"]))
PY
```

Expect: `chunks: 17`, 9 current ids, and 8 old ids, all ending in `-2024` or `-2025`.

Negative test: run Level 1 on a collection ingested *before* Step 8, for example by checking out `8c97952` and re-ingesting. It reports `FAIL (18)`: no metadata, and the where-filter returns 0 chunks. The rebuild script does this automatically.

Evidence: commit `65ee770`; `runs/step08-chroma-metadata/` contains `chroma_before_reingest.log` (FAIL 18), `ingest.log`, `b1_check_chroma.log` (PASS), `b1_eval_off.log` and `baseline_filter_off.json` (identical to Step 7), and `inspect_chroma.log`.

Steps 9–10: The filter, behind an OFF/ON switch ✅

Why: This is the one new stage B1 adds. Superseded chunks are removed *before* similarity ranking, so they can't compete. It's structural, not a prompt saying "prefer the newest document". The switch keeps the original query available, which is what makes the comparison a real ablation.

What changed in code (commit `3e269fd`), all in `rag.py`:

```
def retrieve(query, k=4, version_aware=False):
    ...
    if version_aware:      # B1 ON: only current chunks are eligible
        res = collection().query(query_embeddings=[qv], n_results=k,
                                 where={"superseded": False},
                                 include=["documents", "metadatas", "distances"])
    else:                  # B1 OFF: the original query, untouched
        res = collection().query(query_embeddings=[qv], n_results=k,
                                 include=["documents", "metadatas", "distances"])
```

`answer()` passes the flag through, and `rag.py ask` gained `--version-aware`. The embedder, query text, k, chunking and prompt are identical in both modes.

Do:

```
export RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka
python simple_rag/b1_eval.py --off --out /tmp/b1check   # Test 1: must equal the Step 7 baseline
python simple_rag/b1_eval.py --on  --out /tmp/b1check   # Test 2: preview (the formal AFTER run is
Step 11)
python simple_rag/rag.py ask "Broker 4's data volume just hit 87%. Should that have paged someone?"
python simple_rag/rag.py ask --version-aware "Broker 4's data volume just hit 87%. Should that have
paged someone?"
```

Results (2026-10-18):

Test	Result
1. OFF after the change	identical to Step 7: same 21 cases, same files and ranks, 48.8%
2. ON preview	**0/84 = 0.0% superseded**; superseded 9/9, exact_string 7/7, multi_hop 3/3; 146.9 ms
3. `ask` OFF	retrieved `disk-capacity.md` #1 **and** `disk-capacity-2025.md` #2 (the 90% rule)
3. `ask` ON	the 2025 file is gone; only current runbooks are in the context

An honest note on Test 3: the LLM answered correctly (85%) in *both* modes, because the current runbook ranked #1. With the filter OFF, the outdated 90% rule was still in the context it read. B1 is measured on retrieval for exactly this reason: a right answer built on stale context is luck, not design.

How to compare OFF and ON yourself, case by case:

```
python - <<'PY'
import json
a=json.load(open("/tmp/b1check/baseline_filter_off.json"));
b=json.load(open("/tmp/b1check/version_filter_on.json"))
for x,y in zip(a["cases"],b["cases"]):
    print(x["id"], x["type"].ljust(13), "OFF", "PASS" if x["pass"] else ("-" if x["pass"] is None else
"FAIL"),
          "-> ON", "PASS" if y["pass"] else ("-" if y["pass"] is None else "FAIL"))
PY
```

Evidence: commit `3e269fd`; `runs/step09-10-filter/` contains `b1_eval_off.log` and its JSON (identical to Step 7), `b1_eval_on_preview.log` and its JSON, and `ask_off.log` / `ask_on.log`.

Step 11: AFTER run ✅

Why: This is the measurement the whole project exists for: the same 21 frozen cases, filter ON, committed.

Do:

```
shasum simple_rag/golden_kafka.jsonl          # must still be 850c67f2... (unchanged since the freeze)
python simple_rag/b1_eval.py --on             # writes results/version_filter_on.json
git add simple_rag/results/version_filter_on.json
git commit -m "B1 Track B step 11: AFTER run (filter ON) on the same frozen golden set"
```

Result: 0/84 = **0.0%** superseded chunks; superseded 9/9, exact_string 7/7, multi_hop 3/3. All 19 scored cases pass. The 2 unanswerable cases aren't scored on retrieval. **Evidence:** commit `fbc4a92`; `runs/step11-after/`.

Step 12: Ablation table, fair latency and report ✅

Why: This turns two result files into a result someone else can read and trust.

What changed: `b1_eval.py` gained `--latency N`, which times OFF and ON alternately in one process (fair), and `--table`, which builds the summary from the *saved* results with no rerun. `results/B1_REPORT.md` is the write-up.

Do:

```
python simple_rag/b1_eval.py --latency 5      # results/latency.json
python simple_rag/b1_eval.py --table          # results/ablation_summary.md
```

The result:

Metric	Filter OFF	Filter ON	Target
Superseded chunks retrieved	**48.8%** (41/84)	**0.0%** (0/84)	0% ✅
Superseded cases passed	0/9	9/9	improve ✅
Exact-string cases passed	7/7	7/7	stay flat ✅
Multi-hop cases passed	0/3	3/3	improve ✅
Median retrieval latency (105 interleaved calls each)	50.2 ms	51.1 ms	small ✅

Conclusion: With similarity alone, 48.8% of retrieved chunks came from superseded runbooks, and every version-sensitive question retrieved the old twin. One query-time condition on version metadata cut that to 0% and fixed all 9 superseded and 3 multi-hop cases, while exact-string retrieval stayed at 7/7 and latency moved by under 1 ms.

Why the latency numbers changed: single runs gave ~144–155 ms, dominated by machine warm-up. Interleaving OFF and ON in one process gives the fair comparison, 50.2 vs 51.1 ms.

Evidence: commit `896d73b`; `results/ablation_summary.md` (with a case-by-case OFF→ON table), `results/latency.json`, `results/B1_REPORT.md` (limits and the guide checklist); `runs/step12-report/`.

---

6. Test reference

What	Command (from `track-b-kafka/`)	Pass criterion	Available from
All data checks	`python simple_rag/b1_check.py`	ends with `PASS`	Step 3
Fast data checks	`python simple_rag/b1_check.py --no-trap`	ends with `PASS`	Step 3
Golden set frozen	`git log --oneline -- simple_rag/golden_kafka.jsonl`	one commit, older than the `rag.py` change	Step 6
BEFORE	`python simple_rag/b1_eval.py --off`	41/84 = 48.8% superseded; exact_string 7/7	Step 7
AFTER / both	`python simple_rag/b1_eval.py --on` / no flag	superseded 0% ON; exact-string ON = OFF	Step 11 / 12
Metadata in Chroma	`python simple_rag/b1_check.py --no-trap --chroma`	17/17 chunks tagged; 8 vs 8 files; PASS	Step 8
Look inside Chroma	`python simple_rag/b1_inspect_chroma.py`	filter ON list has no SUPERSEDED row	Step 8
Spot check	`python simple_rag/rag.py ask "<question>"`	answer uses the current value	any time after ingest

Save every run's output into the evidence folder:

```
python simple_rag/b1_check.py 2>&1 | tee $B/b1-tracking/track-b/runs/<stepNN-name>/b1_check.log
```

7. Results log

Date	Step	Result
2026-10-17	2	Unchanged lab: ingest OK, ask OK (21 s), eval retrieval 2/6, answer 2/6
2026-10-17	3	16 runbooks; pair similarity 0.867–0.969; old outranks current 4/8; both versions top-2 for all 8
2026-10-17	4	Manifest: 16 entries, 8 current / 8 superseded, 64 conflicts all verified; negative test caught 3 injected errors (5 FAILs)
2026-10-17	5	21 cases (9/7/3/2); all checks PASS; paraphrase check caught kafka-014, reworded; negative test FAIL 4 as intended
2026-10-18	6	Golden set frozen alone in `2cff45a` (sha1 850c67f2…); no B1 change to rag.py yet
2026-10-18	7	BEFORE (filter OFF): 41/84 = 48.8% superseded chunks; superseded 0/9, exact_string 7/7, multi_hop 0/3; 143.8 ms; rerun identical
2026-10-18	8	Chroma: 17/17 chunks tagged (9 current, 8 superseded); where-filter splits 8/8 files; OFF identical to Step 7 (48.8%); pre-Step-8 collection FAIL (18)
2026-10-18	9–10	Filter + switch: OFF identical to Step 7 (48.8%); ON preview 0/84 = 0.0%, superseded 9/9, exact_string 7/7, multi_hop 3/3, 146.9 ms
2026-10-18	11	AFTER (filter ON, same frozen set): 0/84 = 0.0%; superseded 9/9, exact_string 7/7, multi_hop 3/3
2026-10-18	12	Fair latency (105 calls each) 50.2 vs 51.1 ms; ablation_summary.md + B1_REPORT.md; full 0–12 rebuild 210 s, all fingerprints MATCH
2026-10-18	publish	Commit emails rewritten to the GitHub noreply address (content unchanged); pushed to the public repo (section 11)

8. Troubleshooting

Symptom	Cause	Fix
`No module named pip`	The venv was made by `uv`	`uv pip install <pkg>`
Packages "missing" though installed	Wrong venv activated (an older clone's)	Re-run the session start block; check `which python`
`rag.py: error: unrecognized arguments: --reset`	`ingest` resets by default	Plain `ingest`; `--append` keeps the old collection
`timeout: command not found`	macOS has no GNU `timeout`	Run without it
A proxy call hangs ~150 s	IPv6 route to the proxy	Has not reappeared (21 s on 2026-10-17); retry, or switch networks
`b1_check.py` FAIL on the manifest after editing it	Wrong type (`"3"` vs `3`), bad date, or two current files	Read the FAIL line; fix the entry, or regenerate with `tools/make_corpus.py`
`ask` says the collection is empty	Not ingested, or a different collection	`python simple_rag/rag.py ingest`
torch or numpy import errors after a reinstall	Pins lost	Reinstall with the pins in section 4

9. Glossary

Term	Meaning
Chunk	A piece of a document that gets its own vector. These runbooks are small, so 1 file ≈ 1 chunk
Superseded	A newer version of the same document exists
Golden set	Fixed questions with known right answers, written before the fix
Frozen	Committed to Git before the fix, so it can't be tuned to the result
Ablation	Change one thing (the filter), keep everything else equal, compare
top-k	How many chunks the retriever returns (k = 4 here)
Superseded retrieval rate	Superseded chunks retrieved ÷ all chunks retrieved × 100. The headline metric

10. Regenerating the PDF and the deck

The PDF and the deck in `docs/` are built from this file and two scripts in `tools/docs/`. After updating this SOP for a new step, rebuild them and bump the version (`v2` → `v3`) in the file names.

```
B=~/Desktop/AI-Architect_L2_Aug_2026/project_10082026/b1-tracking/track-b
T=/tmp/b1-docs && mkdir -p $T && cd $T
uv venv -q --python 3.12 .venv && uv pip install -q --python .venv/bin/python markdown
npm init -y >/dev/null && npm install pptxgenjs@3

# SOP -> PDF (headless Chrome)
.venv/bin/python $B/tools/docs/sop_to_html.py $B/SOP.md sop.html $B/tools/docs/flow.html
"/Applications/Google Chrome.app/Contents/MacOS/Google Chrome" --headless=new --no-pdf-header-footer \
  --print-to-pdf="$B/docs/B1_TrackB_Kafka_Version-Aware-Retrieval_SOP_v6_2026-10-18.pdf"
"file://$T/sop.html"

# Deck -> PPTX (slide text lives in build_deck.js; SKILL = the pptx skill folder, for apply_theme.js)
NODE_PATH=$T/node_modules SKILL=<pptx-skill-dir> node $B/tools/docs/build_deck.js \
  "$B/docs/B1_TrackB_Kafka_Version-Aware-Retrieval_Flow_Deck_v6_2026-10-18.pptx"
```

The deck opens in Keynote (File → Open) or PowerPoint. The bar chart on slide 9 is drawn with shapes, not a chart object, because Keynote doesn't render generated chart objects.

---

11. GitHub: where the code lives

Repository (public): https://github.com/shreenathacc22/L2-RAG-B1-track-b-kafka-version-aware

Branch	What's on it	Use it to
`main`	Overview README with links to everything	Start here
`track-b-kafka-version-aware`	The full project: class repo base `6b918c2`, then one commit per B1 step (Steps 3–12), results, `b1-track-b/` (this SOP as PDF and Markdown, run guide, deck, rebuild tools)	Review the work, or rebuild every step
`rag-deployment`	Only what the finished version-aware RAG needs to run, plus `run.sh` and a README	Run the system

Track A (the earlier front-matter version) stays in its own repo: https://github.com/shreenathacc22/L2-RAG-B1-version-aware-retrieval (branch `main`).

Get the code:

```
git clone -b track-b-kafka-version-aware https://github.com/shreenathacc22/L2-RAG-B1-track-b-kafka-version-aware.git b1-track-b-project
git clone -b rag-deployment https://github.com/shreenathacc22/L2-RAG-B1-track-b-kafka-version-aware.git b1-rag-deployment
```

Run the deployment branch:

```
cd b1-rag-deployment
./run.sh setup                          # Python 3.12 venv via uv, installs packages
./run.sh ingest                         # loads the 16 runbooks + metadata into Chroma
./run.sh check                          # data + Chroma metadata checks: PASS
./run.sh eval                           # OFF vs ON: 48.8% -> 0.0%
./run.sh ask "What port is the schema registry on?"     # needs .env (see the README)
```

Rebuild every step from the project branch:

```
cd b1-track-b-project
SKIP_STEP2=1 bash b1-track-b/tools/recreate_track_b.sh ../track-b-rebuild
```

What's never published: `.env` (the API key) and every `.env.*` file are gitignored. `.env.example` was removed, as it was on Track A. Raw run logs aren't published, because they contain local machine paths. Commits use the GitHub noreply address `191985630+shreenathacc22@users.noreply.github.com`, not a personal email.

How it was published (for the record):

```
gh repo create shreenathacc22/L2-RAG-B1-track-b-kafka-version-aware --public --description "..."
git push origin track-b:track-b-kafka-version-aware      # local branch name is track-b
git push origin rag-deployment main
```

Appendix A: Every command, no Claude needed

Set these two variables in every new terminal first:

```
B=~/Desktop/AI-Architect_L2_Aug_2026/project_10082026
T=$B/b1-tracking/track-b/tools
```

A.1 One command: rebuild and verify Steps 0–12

```
bash $T/recreate_track_b.sh                      # builds $B/track-b-rebuild
bash $T/recreate_track_b.sh /some/other/folder   # or choose the folder

# options (put in front of the command)
VENV=$B/mai-practitioner-labs/.venv bash $T/recreate_track_b.sh   # reuse Python env
SKIP_STEP2=1 bash $T/recreate_track_b.sh     # skip lab run (no proxy key)
```

It never touches `track-b-kafka/`.

What it does, in order: clone the class repo at `6b918c2` → branch `track-b` → copy `.env` → Python env → Step 2 lab run → Steps 3–4 corpus, manifest and checks (commit) → Step 5 golden set and checks → both negative tests → Step 6 freeze (commit) → Step 7 ingest Kafka and BEFORE run (commit) → Step 8 Chroma negative test, re-ingest with metadata, `--chroma` check, OFF-unchanged check (commit) → Steps 9–10 filter: OFF still equals baseline, ON preview 0% (commit) → Step 11 AFTER = 0.0% · 9/9 · 7/7 · 3/3 (commit) → Step 12 latency, table, report (commit) → compare 7 fingerprints. It stops at the first failure. Logs go to `<folder>.logs/`.

Expect the last lines to be:

```
OK   corpus (16 .md)    MATCH (1b9bb81564f052d37a2fdba1fe4f4d986b3a64bc)
OK   manifest.json      MATCH (dea81e27843ffee677863915fa998b1dfaf5ab9c)
OK   golden_kafka.jsonl MATCH (850c67f2eab21aa1a6afabd481e5c3efccb7e0ba)
OK   b1_check.py (st 8) MATCH (4811e119fbbea14f97d5db84e327b653025069ae)
OK   rag.py (step 10)   MATCH (59dbe54f952b5dfef50c1e613479d5325729e955)
OK   b1_eval.py (st 12) MATCH (a815f0d85440ff8c8a367296808ac4d2e0240327)
OK   b1_inspect_chroma  MATCH (a2be529f0b16101dc714003b4b2d8bfd57b2ed1d)
== Done · Steps 0-12 recreated in .../track-b-rebuild
```

Tested 2026-10-18 (Steps 0–12): a full run on this Mac took 210 s: AFTER 0.0% · 9/9 · 7/7 · 3/3, and all 7 fingerprints MATCH. All checks were OK, all three negative tests FAILed as intended (manifest, golden, Chroma before re-ingest), BEFORE was 48.8%, OFF was unchanged after Step 8, and all 7 fingerprints matched.

To start over: `rm -rf $B/track-b-rebuild $B/track-b-rebuild.logs`

A.2 Command by command

Run these from a new folder. Each block lists what you should see.

Step 0: get the code

```
cd $B
git clone https://github.com/balajivis/mai-practitioner-labs.git track-b-kafka
cd track-b-kafka
git checkout 6b918c2                 # the class-repo commit this was built on
git switch -c track-b
cp $B/.env .env && chmod 600 .env    # class proxy key; .gitignore keeps it out of git
git branch --show-current            # expect: track-b
```

Python environment (once)

```
uv venv --python 3.12 .venv
uv pip install --python .venv/bin/python -r requirements.txt \
   "torch==2.2.2" "numpy<2" "transformers<5" "sentence-transformers<6"
source .venv/bin/activate
which python                         # expect: .../track-b-kafka/.venv/bin/python
```

(The original build reused `$B/mai-practitioner-labs/.venv`. Either works: `source` that one instead.)

Step 2: unchanged lab

```
python simple_rag/rag.py ingest                      # expect: vectors stored ~196
python simple_rag/rag.py ask "how do I tow my car?"  # expect: an answer with [1] [2] citations
python simple_rag/rag.py eval                        # expect: retrieval 2/6 · answer 2/6
```

Steps 3–4: corpus, manifest and data checks

```
python $T/make_corpus.py simple_rag/corpus/kafka     # expect: wrote 16 runbooks + manifest.json
cp $T/source/b1_check.py simple_rag/b1_check.py
python simple_rag/b1_check.py                        # expect: old version outranks current for 4/8
... PASS
git add simple_rag/corpus/kafka simple_rag/b1_check.py
git commit -m "B1 Track B steps 3-4: versioned Kafka corpus, manifest and data checks"
```

Step 5: golden set

```
python $T/make_golden.py simple_rag/golden_kafka.jsonl   # expect: wrote 21 cases
python simple_rag/b1_check.py --no-trap                  # expect: [Step 5] ... PASS
```

Negative tests: the checks must FAIL on broken data

```
M=simple_rag/corpus/kafka/manifest.json; G=simple_rag/golden_kafka.jsonl
cp $M /tmp/manifest.bak; cp $G /tmp/golden.bak

python - <<'PY'
import json; p="simple_rag/corpus/kafka/manifest.json"; m=json.load(open(p))
m["files"]["disk-capacity-2025.md"]["superseded"]=False
m["files"]["topic-retention.md"]["version"]="3"
m["files"]["schema-registry.md"]["effective_date"]="2026-3-20"
json.dump(m,open(p,"w"),indent=2)
PY
python simple_rag/b1_check.py --no-trap; echo "exit=$?"   # expect: FAIL (...) and exit=1
cp /tmp/manifest.bak $M

python - <<'PY'
import json; p="simple_rag/golden_kafka.jsonl"; rows=[json.loads(l) for l in open(p)]
rows[0]["expect"]["must_cite"]=[{"doc":"Kafka Broker Disk Capacity Runbook","version":2}]
rows[2]["expect"]["must_not_contain"].append("3 days")
open(p,"w").write("".join(json.dumps(r)+"\n" for r in rows))
PY
python simple_rag/b1_check.py --no-trap; echo "exit=$?"   # expect: FAIL (4) and exit=1
cp /tmp/golden.bak $G
python simple_rag/b1_check.py --no-trap | tail -1         # expect: PASS
```

The FAIL count for the manifest test depends on the step: 5 at Step 4, and 14 once the golden set exists, because broken metadata also breaks the golden-set checks. Any FAIL with exit=1 is correct.

Step 6: freeze

```
git add simple_rag/golden_kafka.jsonl
git commit -m "Freeze B1 golden set before version-aware retrieval"
git log --oneline -- simple_rag/golden_kafka.jsonl   # expect: exactly one commit
git log --oneline -1 -- simple_rag/rag.py            # expect: 00fe688 (upstream), no B1 change yet
```

Step 7: evaluator and BEFORE run

```
cp $T/source/rag.py simple_rag/rag.py            # adds RAG_CORPUS / RAG_COLLECTION; retrieval
unchanged
cp $T/source/b1_eval.py simple_rag/b1_eval.py
export RAG_CORPUS=simple_rag/corpus/kafka RAG_COLLECTION=kafka
python simple_rag/rag.py ingest --formats md     # expect: vectors stored 17
python simple_rag/b1_eval.py --off               # expect: superseded chunks retrieved 41/84 = 48.8%
git add simple_rag/rag.py simple_rag/b1_eval.py simple_rag/results/baseline_filter_off.json
git commit -m "B1 Track B step 7: evaluator and BEFORE baseline (filter OFF)"
```

Step 8: version metadata into Chroma

```
cp $T/source/step08/rag.py $T/source/step08/b1_check.py $T/source/step08/b1_inspect_chroma.py
simple_rag/
python simple_rag/b1_check.py --no-trap --chroma  # BEFORE re-ingest: expect FAIL (18), the negative
test
python simple_rag/rag.py ingest --formats md      # re-ingest: metadata goes in
python simple_rag/b1_check.py --no-trap --chroma  # expect: PASS (17 chunks, 8 vs 8 files)
python simple_rag/b1_eval.py --off                # expect: still 41/84 = 48.8%
python simple_rag/b1_inspect_chroma.py            # look inside + filter demo
git add simple_rag/rag.py simple_rag/b1_check.py simple_rag/b1_inspect_chroma.py
git commit -m "B1 Track B step 8: version metadata attached to every chunk in Chroma"
```

Steps 9–10: filter and switch

```
cp $T/source/step10/rag.py simple_rag/rag.py
python simple_rag/b1_eval.py --off --out /tmp/b1check   # expect: 41/84 = 48.8% (unchanged)
python simple_rag/b1_eval.py --on  --out /tmp/b1check   # expect: 0/84 = 0.0%, exact_string 7/7
python simple_rag/rag.py ask --version-aware "Broker 4's data volume just hit 87%. Should that have
paged someone?"
git add simple_rag/rag.py
git commit -m "B1 Track B steps 9-10: query-time version filter behind an OFF/ON switch"
```

Step 11: AFTER run

```
python simple_rag/b1_eval.py --on                 # expect: 0/84 = 0.0%, 9/9, 7/7, 3/3
git add simple_rag/results/version_filter_on.json
git commit -m "B1 Track B step 11: AFTER run (filter ON) on the same frozen golden set"
```

Step 12: latency, table, report

```
cp $T/source/step12/b1_eval.py simple_rag/b1_eval.py
python simple_rag/b1_eval.py --latency 5          # expect: OFF ~50 ms, ON ~51 ms
python simple_rag/b1_eval.py --table              # expect: results/ablation_summary.md
cp $T/source/step12/B1_REPORT.md simple_rag/results/B1_REPORT.md
git add simple_rag/b1_eval.py simple_rag/results/
git commit -m "B1 Track B step 12: ablation table, fair latency and report"
```

A.3 Fingerprints of a correct build

```
cat simple_rag/corpus/kafka/*.md | shasum      # 1b9bb81564f052d37a2fdba1fe4f4d986b3a64bc
shasum simple_rag/corpus/kafka/manifest.json   # dea81e27843ffee677863915fa998b1dfaf5ab9c
shasum simple_rag/golden_kafka.jsonl           # 850c67f2eab21aa1a6afabd481e5c3efccb7e0ba
shasum simple_rag/b1_check.py                  # e9a0ca72... after Step 3-7,
4811e119fbbea14f97d5db84e327b653025069ae after Step 8
shasum simple_rag/rag.py                       # 5874d262... Step 7, a5587411... Step 8,
59dbe54f952b5dfef50c1e613479d5325729e955 Steps 9-10
shasum simple_rag/b1_eval.py                   # 92a9b53c... Steps 7-11,
a815f0d85440ff8c8a367296808ac4d2e0240327 Step 12
shasum simple_rag/b1_inspect_chroma.py         # a2be529f0b16101dc714003b4b2d8bfd57b2ed1d
```

If one differs, `diff` it against the original in `$B/track-b-kafka/` to see what changed.